

路由Routing



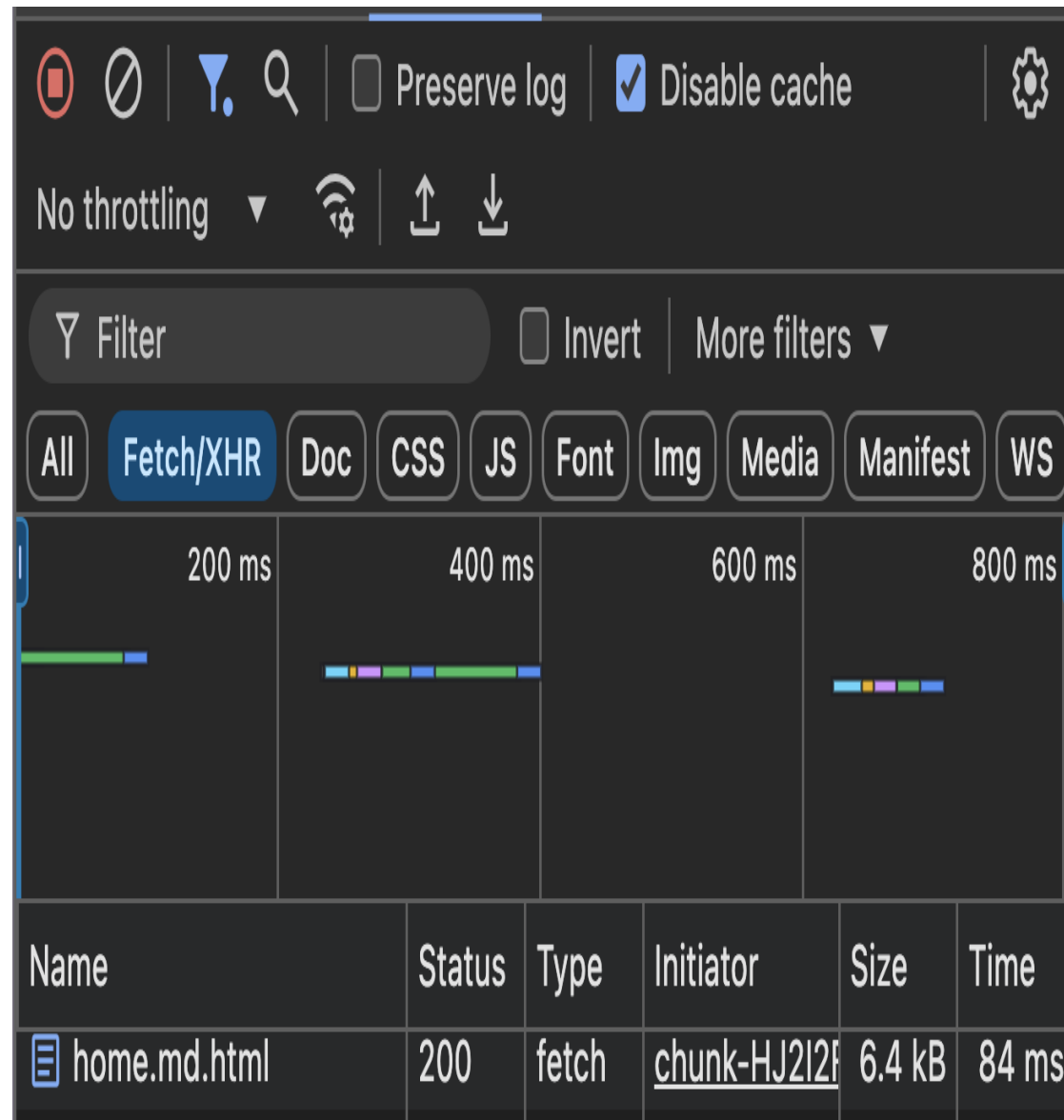
什麼是Routing?

在講解Angular Routing之前，先來了解一下瀏覽器的流程。

URL > HTTP > HTML > render

當我們輸入URL按確定之後，瀏覽器會向服務端發送HTTP請求。

服務端收到HTTP請求之後會返回HTML格式的內容，當瀏覽器收到HTML格式的內容後就會將其渲染出畫面。



什麼是Routing?

在單頁應用程式中，您可以透過顯示或隱藏與特定元件相對應的顯示部分來變更使用者所看到的內容，而不是前往伺服器以取得新頁面。

當使用者執行應用程式任務時，他們需要在您定義的不同視圖之間移動。

要處理從一個視圖到下一個視圖的導航，您可以使用 Angular Router。Router透過將瀏覽器 URL 解釋為更改視圖的指令來啟用導航。

如何使用Routing?

首先先建立一個專案，並且新增兩個組件first與second。

```
lingyilun@lingyilundeMacBook-Pro-10 % cd demo4  
lingyilun@lingyilundeMacBook-Pro demo4 % ng g c first  
CREATE src/app/first/first.component.scss (0 bytes)  
CREATE src/app/first/first.component.html (20 bytes)  
CREATE src/app/first/first.component.spec.ts (585 bytes)  
CREATE src/app/first/first.component.ts (231 bytes)  
lingyilun@lingyilundeMacBook-Pro demo4 % ng g c second  
CREATE src/app/second/second.component.scss (0 bytes)  
CREATE src/app/second/second.component.html (21 bytes)  
CREATE src/app/second/second.component.spec.ts (592 bytes)  
CREATE src/app/second/second.component.ts (235 bytes)  
lingyilun@lingyilundeMacBook-Pro demo4 %
```

如何使用Routing?

接著來到app.routes.ts中，並且將其import進此頁面，看到他反灰的不用緊張只是因為我們還沒有使用到。

```
import { Routes } from '@angular/router';  
import { FirstComponent } from './first/first.component';  
import { SecondComponent } from './second/second.component';
```



如何使用Routing?

到這裡時我們要先確認一下我們的路由是否有被正確導入，請打開你的app.config.ts文件並確認是有有將routes新增進provideRouter函數之中，正常來說系統預設就會將其設定好，除非後面手動新增就需要自己處理。

```
o4 > src > app > TS app.config.ts > ...
import { ApplicationConfig, provideZoneChangeDetection } from '@angular/core';
import { provideRouter } from '@angular/router';

import { routes } from './app.routes';

export const appConfig: ApplicationConfig = {
  providers: [provideZoneChangeDetection({ eventCoalescing: true }), provideRouter(routes)]
};
```

定義基本路線

創建路線基本由三個基礎建構組成。

- 為你的路線設定一個數組(正常系統會自動新增)。
- 在數組中定義你的路線。
- 把你設定得路線加進你的程式中(畫面或者TS)

為你的路線設定一個數組

這部分當你建立專案時Angular CLI就已經幫你建立完成了，處於一個基礎無設定的狀態。

```
import { Routes } from '@angular/router';  
  
export const routes: Routes = [];
```


在數組中定義你的路線

這邊就是根據你的需求來去設定，path設定的名稱為網址顯示的名稱而component就是這個網址需要對應到哪個組件畫面，我們就拿一開始新建的兩個組件來做示範。

```
import { FirstComponent } from './first/first.component';
import { SecondComponent } from './second/second.component';

export const routes: Routes = [
  { path: 'first', component: FirstComponent },
  { path: 'second', component: SecondComponent }
];
```

錯誤頁面

當你的path設定為**時，當你的URL為不存在的路徑時他將會把你導至你設定的頁面(注意需寫在router.ts的最下面，否則上面會無法顯示內容)。

```
{ path: '**', component: PageNotFoundComponent },
```

設定重新導向

如果你希望使用者連到特定URL時要切換到某個頁面，你就可以使用這個方法，下圖的寫法是如果使用者輸入的URL沒有後面網址就將其重新轉移至/first頁面，pathMatch有兩種寫法full跟prefix，full是指URL需與你設定的path完全一致才會重新導向，而prefix是只要你的URL已指定路徑開頭就會重新導向(例如path設定bbb，URL:為http://localhost:4200/bbb/aaa)。

```
{ path: '', redirectTo: '/first', pathMatch: 'full' }, // redirect to `first-component`
```

嵌套路由

嵌套路由(又稱子路由)就是你想要見另一個與根元件以外的元件相關的路由，簡單來說我一開始已經有了一個child-a的元件在頁面上使用了，我想要在first裡面有下一層路由(根元件 > first > child-a)。

```
export const routes: Routes = [
  { path: 'first', component: FirstComponent,
    children: [{
      path: 'child-a', component: ChildAComponent
    }]
  },
];
```

將你設定得路線加進你的程式中

當你設定完路線後，你就需要有個地方可以讓你切換頁面，有兩種辦法通常除非需要做判斷後才換頁否則都會是寫在HTML中讓使用者去切換頁面。

- 使用HTML導航
- 使用TS寫程式切換

使用HTML導航

要使用頁面導航時請先將RouterOutlet, RouterLinkActive, RouterLink這三個模板加入該畫面的import中，這樣後面才可以在HTML中使用該元件。

```
import { Router, RouterLink, RouterLinkActive, RouterOutlet } from '@angular/router';

@Component({
  selector: 'app-root',
  standalone: true,
  imports: [RouterOutlet, RouterLinkActive, RouterLink],
  templateUrl: './app.component.html',
  styleUrls: ['./app.component.scss']
})
```

使用HTML導航

RouterOutlet就是畫面上的<router-outlet>，你導航的頁面內容會顯示在這個宣告標籤的地方。

```
<h1>Angular Router</h1>
<nav>
  <ul>
    <li><a routerLink="/first" routerLinkActive="active">First Component</a></li>
    <li><a routerLink="/second" routerLinkActive="active">Second Component</a></li>
  </ul>
</nav>
<button routerLink="/first" routerLinkActive="active">First Component</button>
<h3 routerLink="/first" routerLinkActive="active">First Component</h3>
<!-- 導航頁面的內容會放在這裡 -->
<router-outlet></router-outlet>
```

使用HTML導航

RouterLink是讓HTML有辦法使用routerLink的元件，這個是用來指定要切換的網頁路徑。

```
<h1>Angular Router</h1>
<nav>
  <ul>
    <li><a routerLink="/first" routerLinkActive="active">First Component</a></li>
    <li><a routerLink="/second" routerLinkActive="active">Second Component</a></li>
  </ul>
</nav>
<button routerLink="/first" routerLinkActive="active">First Component</button>
<h3 routerLink="/first" routerLinkActive="active">First Component</h3>
<!-- 導航頁面的內容會放在這裡 -->
<router-outlet></router-outlet>
```


使用HTML導航

RouterLinkActive是讓HTML有辦法使用routerLinkActive的元件，這個是一個CSS的用法，當你的頁面就是跟這個路徑是同一個的時候就會執行這段並且觸發其指定的CSS，RouterLinkActive需配合RouterLink一起使用，否則不會生效。

```
<h1>Angular Router</h1>
<nav>
  <ul>
    <li><a routerLink="/first" routerLinkActive="active">First Component</a></li>
    <li><a routerLink="/second" routerLinkActive="active">Second Component</a></li>
  </ul>
</nav>
<button routerLink="/first" routerLinkActive="active">First Component</button>
<h3 routerLink="/first" routerLinkActive="active">First Component</h3>
<!-- 導航頁面的內容會放在這裡 -->
<router-outlet></router-outlet>
```

```
.active {
  color: red;
}
```

使用HTML導航

只要你有引用RouterOutlet, RouterLinkActive, RouterLink這三個，其實你可以在HTML中每個標籤使用，不論是Button、文字(h1~h6)，不過建議還是要根據操作邏輯使用在<button>按鈕或者是<a>超連結中。

```
<h1>Angular Router</h1>
<nav>
  <ul>
    <li><a routerLink="/first" routerLinkActive="active">First Component</a></li>
    <li><a routerLink="/second" routerLinkActive="active">Second Component</a></li>
  </ul>
</nav>
<button routerLink="/first" routerLinkActive="active">First Component</button>
<h3 routerLink="/first" routerLinkActive="active">First Component</h3>
<!-- 導航頁面的內容會放在這裡 -->
<router-outlet></router-outlet>
```

使用HTML導航

Angular Router

- [First Component](#)
- [Second Component](#)

First Component

First Component

first works!

Angular Router

- [First Component](#)
- [Second Component](#)

First Component

First Component

second works!

使用TS導航

當你切換頁面需要根據使用者操作或者資料變化而改變切換的頁面時，你就需要在TS中用程式去執行切換頁面的動作，第一步請先將Router注入此頁面的建構式也就是constructor中。

```
import { Component, OnInit } from '@angular/core';  
import { Router, RouterLink, RouterLinkActive, RouterOutlet } from '@angular/router';
```

```
constructor(private router: Router) {}
```

使用TS導航

緊接個你就可以使用this.router中的指令去切換頁面，下圖的兩種寫法都是可行的，差異只在一個需要用中括弧去將其路徑包起來一個不用。

```
ngOnInit() {  
  this.router.navigate(['/first']);  
  this.router.navigateByUrl('/first');  
}
```

練習

路由練習

使用HTML routerLink做觸發換頁

[First Component](#)

[Second Component](#)

按鈕需使用TS做觸發換頁

返回起始頁

First Component

Second Component

進階練習

新增一個子路由並且讓他顯示

路由練習

使用HTML routerLink做觸發換頁

[First Component](#)

[Second Component](#)

按鈕需使用TS做觸發換頁

起始頁

First Component

Second Component

Child-a Component

First Component內容

我是first頁面底下的child-a

網址帶值

正常情況下頁面跟頁面切換我們會用service去傳遞內容，但其實還有一種方式可以將值塞在網址上，這種方式有兩種寫法分別是

router.ts中設定路由要帶的值的名稱

直接在網址後面加上?名稱跟內容

這兩種方法都可以讓你在網址切換的時候不需要透過service去傳遞內容，當然正常情況下不會在網址中放太多或者太複雜的值，大部分可能只是某個問卷的ID或者使用者的名稱之類的內容。

網址帶值(router.ts)

首先來提到我們要怎麼寫在router.ts中，假設我現在有個頁面叫做list，我們會在router.ts中去設定他的網址跟對應的畫面(如左下)，那這個時候如果在進到這個頁面的時候想要傳遞一個值(假設是使用者名稱Allen)，那這個時候你就可以多一條router並且在路徑list後面加上/:name，這格時候你就會有兩條網址設定(如右下)，網址會變成list/Allen，也可以帶多個值(list/:name/:age)。

```
{ path: 'list', component: ListComponent },
```

```
{ path: 'list', component: ListComponent },  
{ path: 'list/:name', component: ListComponent },
```

網址帶值(router.ts)

緊接著我們就可以去頁面撈取你帶如該頁面的值(目前網址為list/Allen)，首先要抓取網址中的值需要注入一個套件ActivatedRoute，所以要將其寫在建構式中(因為是要用TS抓取)，並且抓取值得寫法為snapshot.paramMap.get('設定路徑的值的名稱')。

```
constructor(private activatedRoute: ActivatedRoute) {}
```

```
ngOnInit(): void {  
  console.log(this.activatedRoute.snapshot.paramMap.get('name'));  
}
```

網址帶值(url)

第二種寫法你不需要動到router.ts的內容只需要修改你的網址，同樣拿list頁面為例，我們要傳遞的值一樣是使用者名稱Allen，那我的網址就會寫成list?

name=Allen。

緊接著一樣要注入ActivatedRoute，但是抓取的時候的寫法跟router.ts的寫法稍微不同，然後可以一次帶多個值寫法為?name=Allen&age=12，但是注意上面的值皆為string格式。

```
ngOnInit(): void {  
  console.log(this.activatedRoute.snapshot.queryParamMap.get('name'));  
}
```

结束

